



Lesson 41

24.11.2025



Spring Cloud

Spring Cloud надає інструменти, які дозволяють розробникам швидко створювати поширені патерни для розподілених систем (наприклад, керування конфігурацією, виявлення сервісів, circuit breaker'и, інтелектуальна маршрутизація, control bus, короткоживучі мікросервіси та контрактне тестування).

Координація розподілених систем часто призводить до появи великої кількості шаблонного коду, і завдяки Spring Cloud розробники можуть швидко розгорнути сервіси й застосунки, що реалізують ці патерни.

Вони чудово працюють у будь-якому розподіленому середовищі — включно з локальним ноутбуком розробника, фізичними дата-центрами та керованими платформами, такими як Cloud Foundry.



Spring Cloud зосереджується на тому, щоб забезпечити зручну роботу «з коробки» для типових сценаріїв використання та надати механізми розширюваності для покриття інших випадків.

- Distributed/versioned configuration
- Service registration and discovery
- Routing
- Service-to-service calls
- Load balancing
- Circuit Breakers
- Distributed messaging
- Short lived microservices (tasks)
- Consumer-driven and producer-driven contract testing

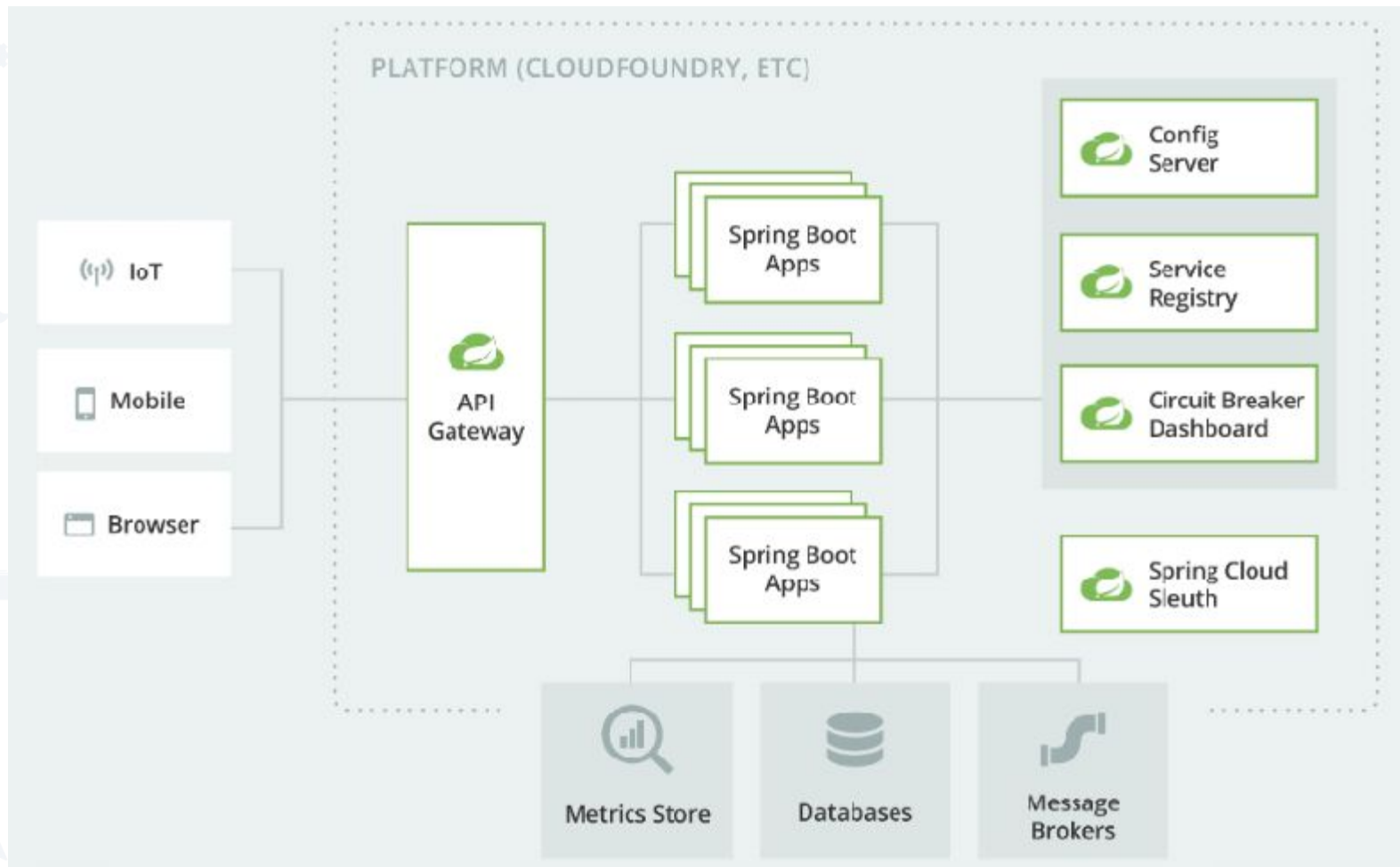


NETFLIX
OSS



Spring
Cloud

Компонент / Функція	Netflix OSS	Spring Cloud	Коментар
Service Discovery	Eureka	Spring Cloud Eureka	Spring Cloud значно гнучкіший
Load Balancing	Ribbon (<i>deprecated</i>)	Spring Cloud Load Balancer	Сучасна заміна Ribbon
Circuit Breaker / Fault Tolerance	Hystrix (<i>deprecated</i>)	Resilience4j	Resilience4j кращий за продуктивністю та фічами
API Gateway	Zuul (<i>deprecated</i>)	Spring Cloud Gateway	Gateway — реактивний і активно підтримується
Configuration Management	Archaius	Spring Cloud Config	Spring Cloud = cloud-native підхід
HTTP Client	Feign	OpenFeign	Концептуально однакові; OpenFeign сучасніший





Service registration and discovery

Service Registry (Реєстр сервісів)

Service Registry — це центральний каталог усіх доступних мікросервісів, у якому містяться:

- логічна назва сервісу,
- IP-адреса,
- порт,
- статус (працює / не працює),
- додаткові метадані.

Кожен мікросервіс **при запуску реєструється** в реєстрі та періодично надсилає сигнал «я живий» (heartbeat)

Service Discovery (Виявлення сервісів)

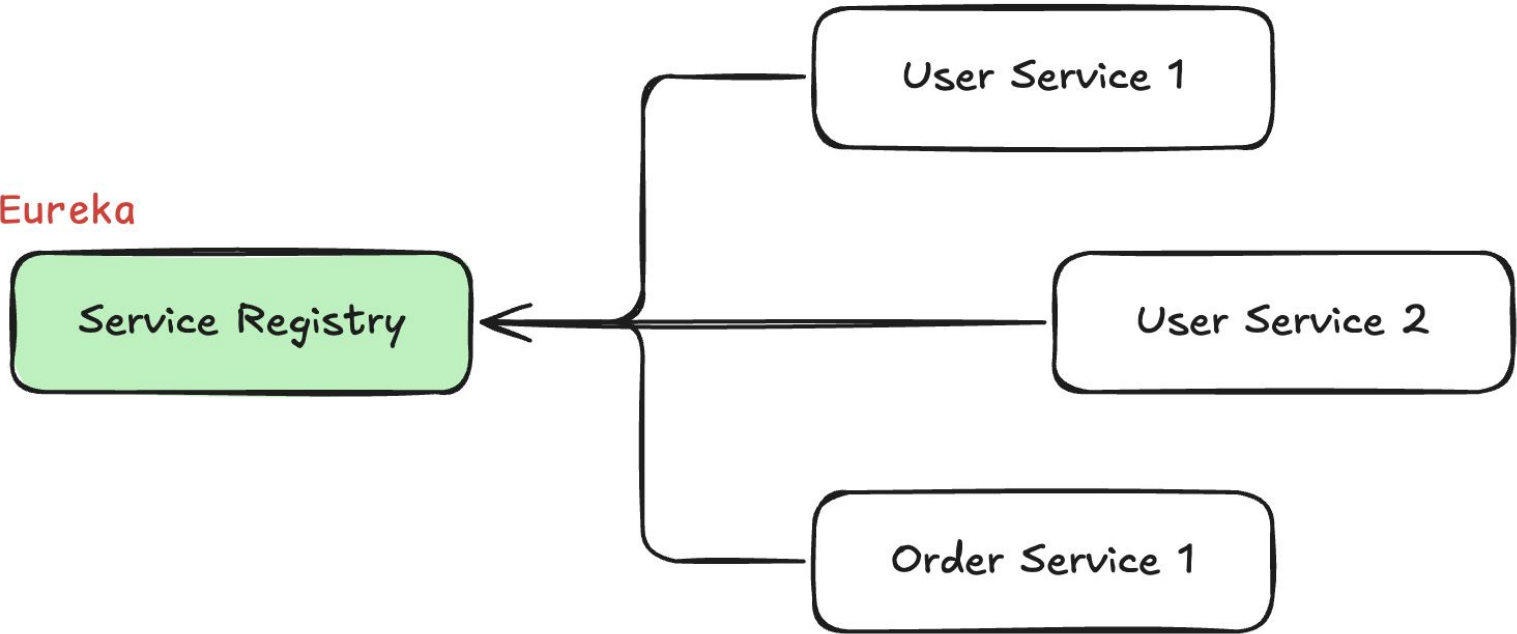
Service Discovery — це механізм автоматичного пошуку сервісів через **Service Registry**.

Тобто сервіс А не знає, на якій машині розташований сервіс В. Він запитує у реєстру:

«Дай адресу сервісу USER-SERVICE».

Реєстр повертає список активних інстансів, а клієнт вибирає один (іноді через балансування навантаження).

Eureka





API Gateway — це єдина точка входу для всіх клієнтських запитів у мікросервісній системі. Він приймає запит від клієнта, визначає, якому мікросервісу цей запит потрібен, і пересилає його далі.

Що робить API Gateway:

Маршрутизація (Routing)

Перенаправляє запити на потрібний мікросервіс (наприклад, `/users/**` → User Service).

Балансування навантаження

Вибирає правильний інстанс сервісу, якщо їх декілька.

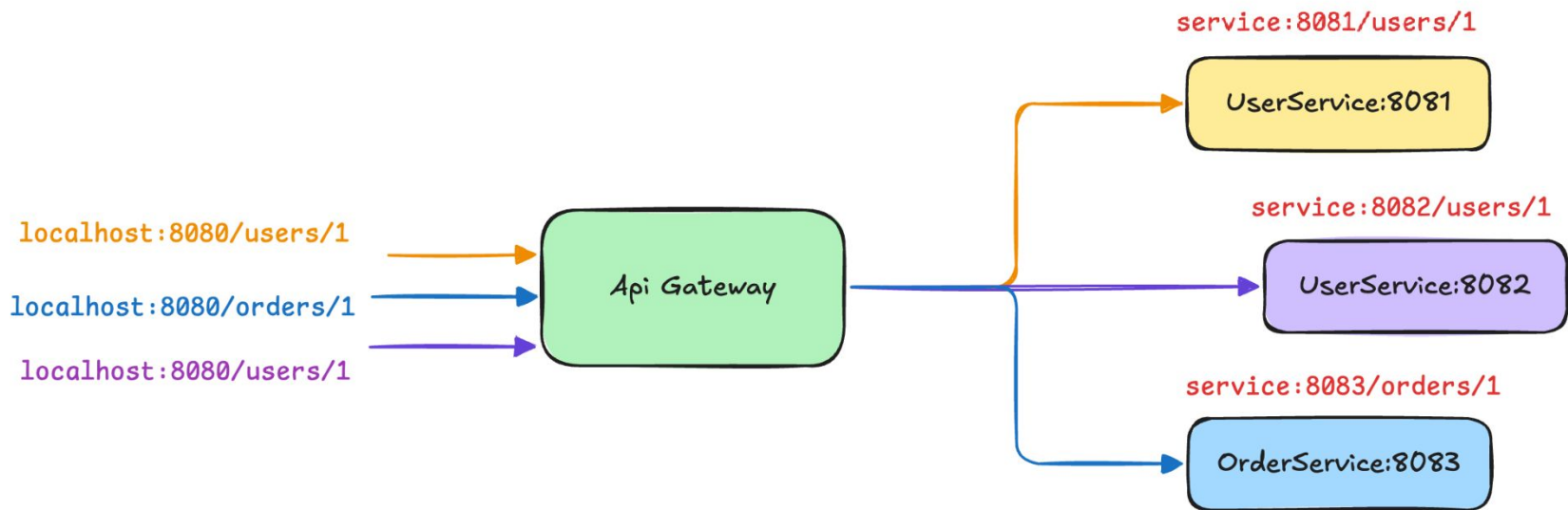
Агрегація відповідей

Може об'єднувати дані з кількох сервісів в один результат.

Фільтри / Middleware

Обробка запитів і відповідей:

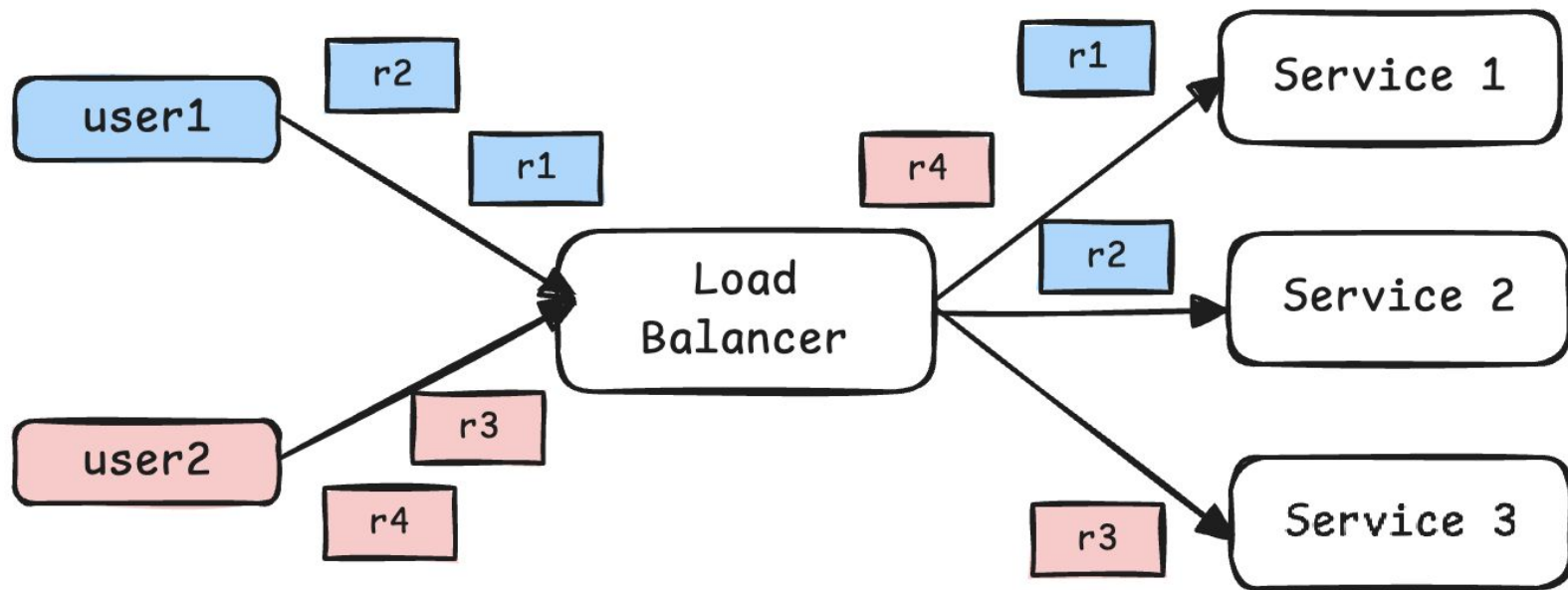
- логування
- авторизація / аутентифікація
- перехоплення помилок
- обмеження швидкості (rate limiting)
- кешування



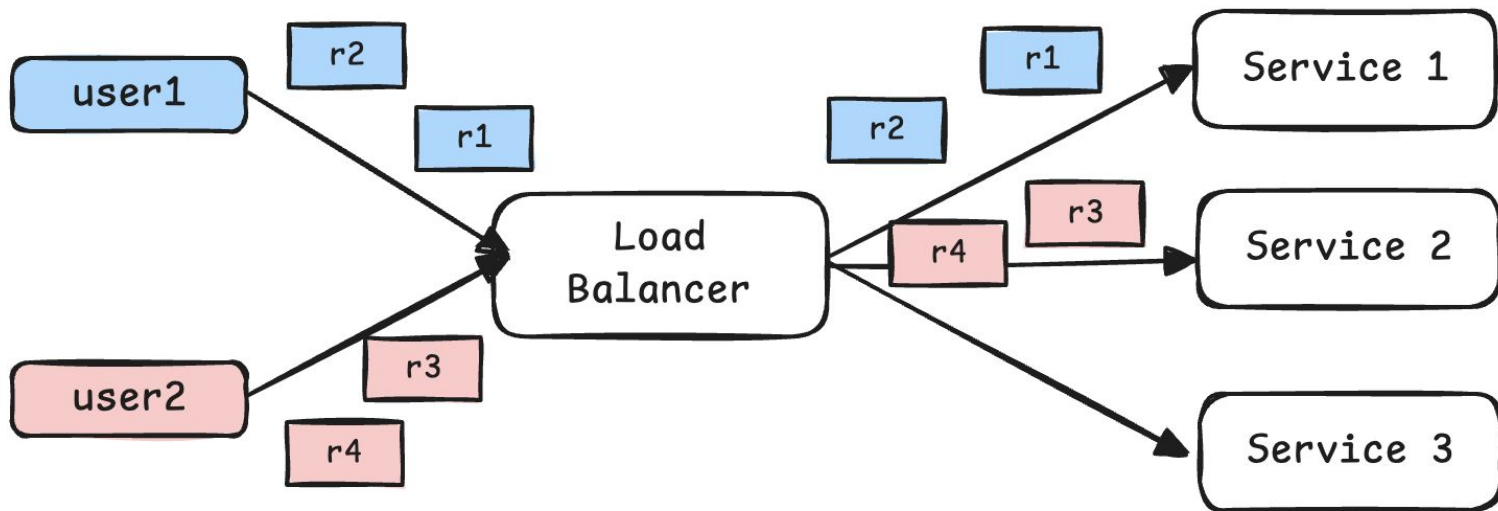


Load-balancing algorithms

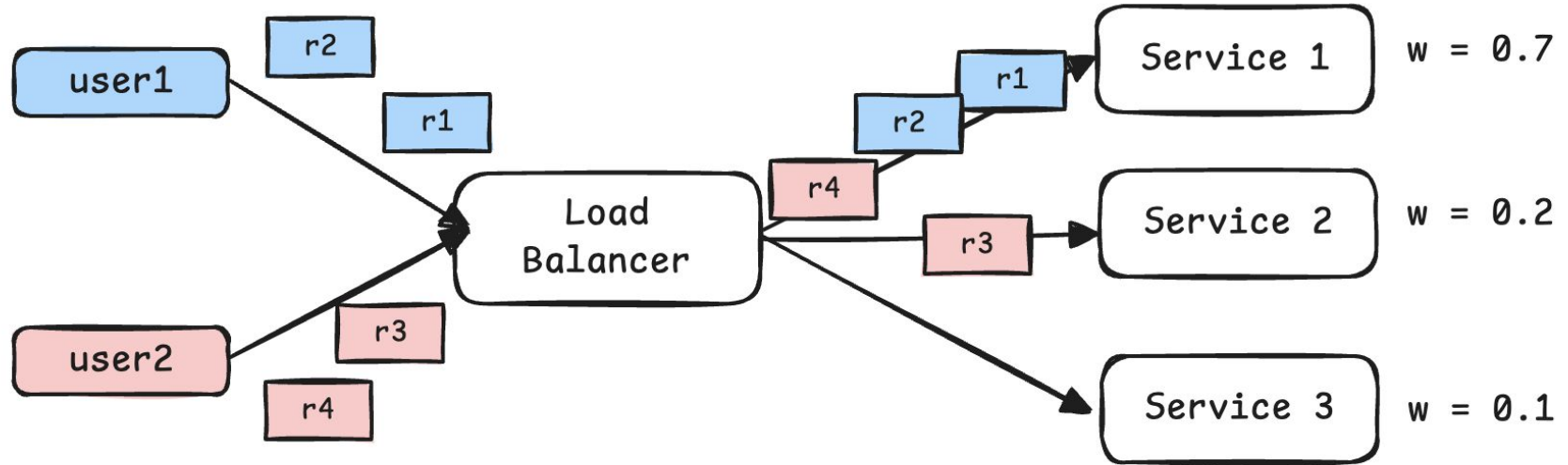
1. Round robin



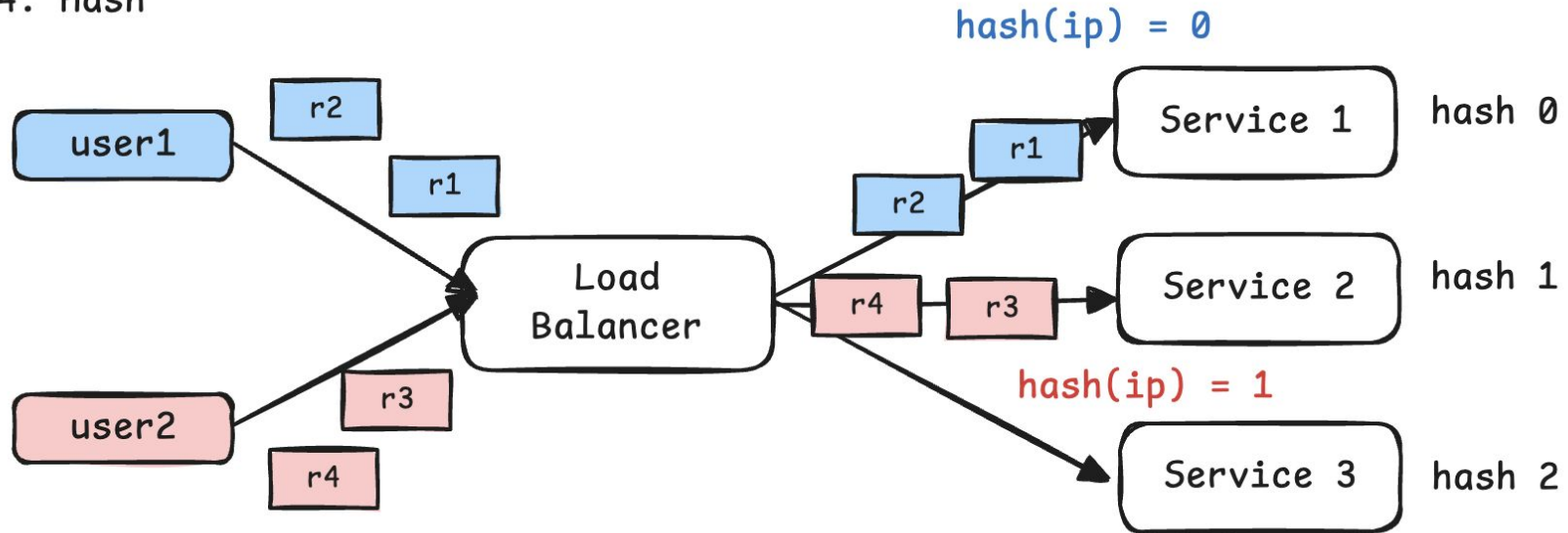
2. Sticky round-robin



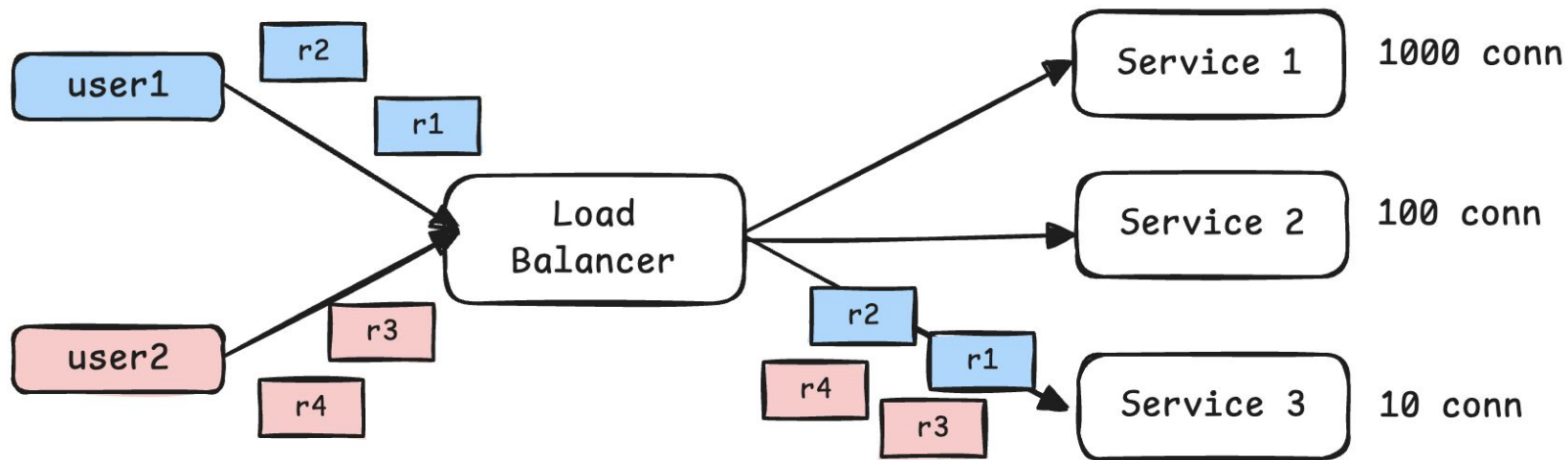
3. Weighted round-robin



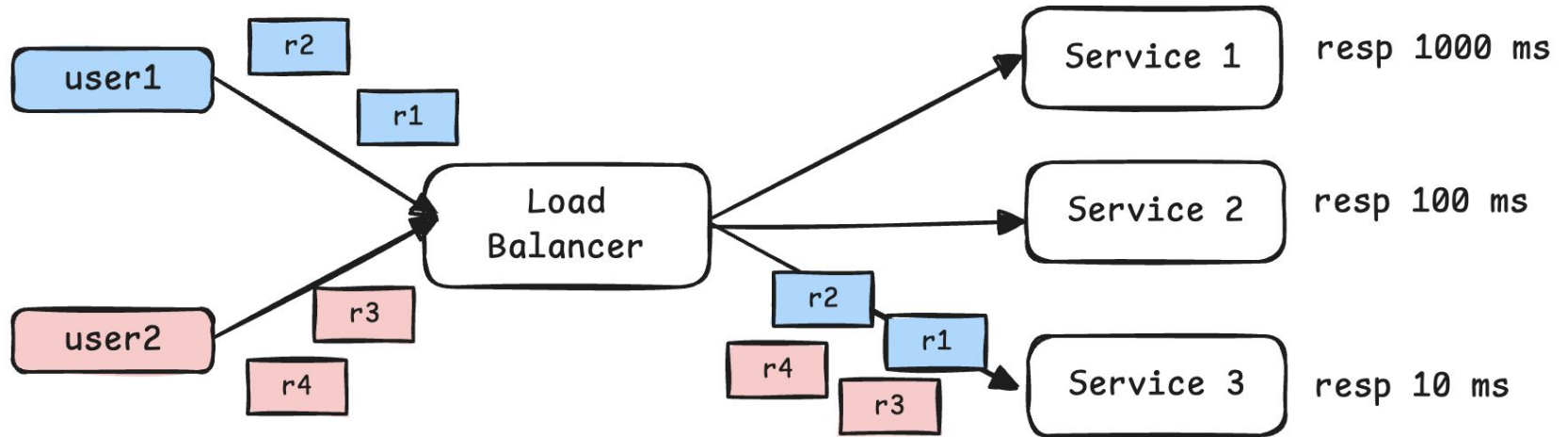
4. Hash



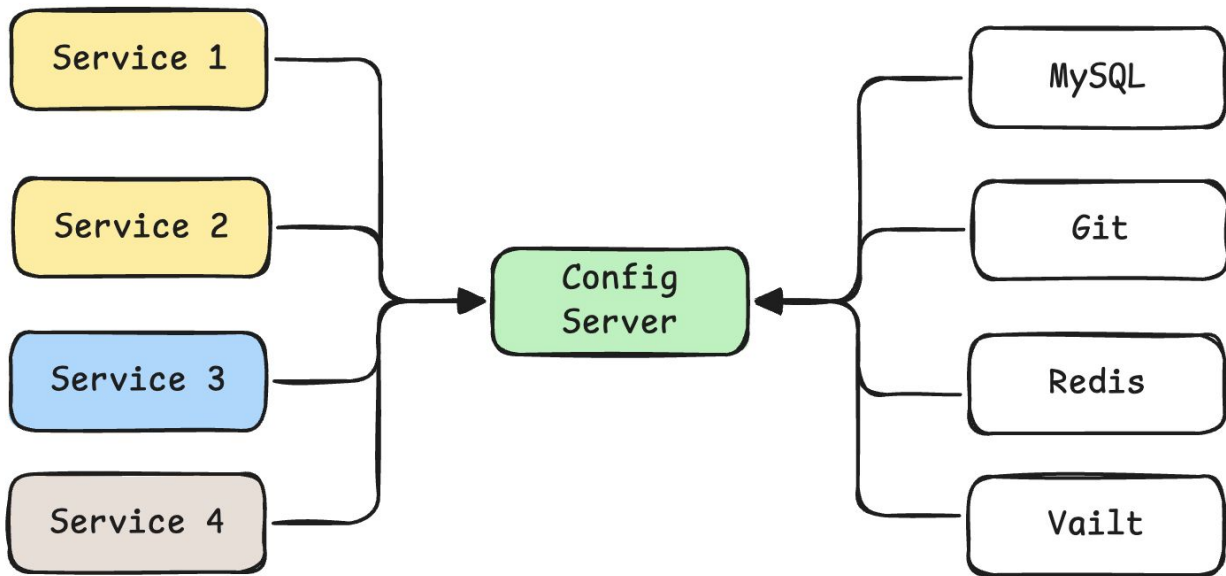
5. Least connections



6. Least response time



Configuration Management (керування конфігурацією) — це підхід, який дозволяє централізовано керувати налаштуваннями для всіх мікросервісів. У Spring Cloud це робиться через **Spring Cloud Config Server**.



Spring Cloud OpenFeign — це бібліотека, яка дозволяє викликати інші мікросервіси так, ніби ти викликаєш звичайні Java-методи.

По суті, це **декларативний HTTP-клієнт**: ти просто описуєш інтерфейс → Feign генерує реалізацію і під капотом робить HTTP-запити.

